Frontend dokumentáció

FRONTEND DOKUMENTÁCIÓ

Planova

A "*"-gal megjelölt elemek, sorok fejlesztés alatt állnak vagy jövőbeli terveket tükröznek.

Contents

Frontend dokumentáció 1
FRONTEND DOKUMENTÁCIÓ 1
Planova 1
1. Projekt áttekintés 2
2. Technológiai stack 3
  2.1. Közös technológiák 3
  2.2. Web alkalmazás (apps/web) 3
  2.3. Mobil alkalmazás (apps/mobile) * 3
  2.4. Fejlesztői eszközök 3
3. Projekt struktúra 3
  3.1. Monorepo felépítés 4
  3.2. Web alkalmazás struktúra (apps/web) 4
  3.3. Mobil alkalmazás struktúra (apps/mobile) * 5
  3.4. Megosztott csomagok (packages/) 5
4. Megosztott csomagok részletes leírása 6
  4.1. @repo/api – API kommunikációs réteg 6
  4.2. @repo/hooks – Állapotkezelés (Zustand store-ok és React Query hook-ok) 6
  4.3. @repo/features – Megosztott feature komponensek 8
  4.4. @repo/ui – Újrafelhasználható UI komponensek 8
  4.5. @repo/types – Típusdefiníciók 8
  4.6. @repo/utils – Segédfüggvények 9
  4.7. @repo/config – Tamagui konfiguráció 9
5. Alkalmazás működése 10
  5.1. Indítás és hitelesítés 10
  5.2. Hitelesítési folyamat 10

5.3. Naptár nézetek ... 11
5.4. Oldalsáv (Sidebar) ... 12
5.5. Naptárbejegyzések ... 12
5.6. Kontextus menü ... 12
5.7. Dialógus ablakok ... 13
5.8. Valós idejű értesítések (SignalR) ... 13
6. Navigációs sáv (Navbar) ... 13
7. Reszponzív működés ... 13
8. Téma rendszer ... 14
9. Provider struktúra ... 15
9.1. Web (Providers.tsx) ... 15
9.2. Mobil (_layout.tsx) * ... 15
10. API végpontok (frontend oldal) ... 15
11. Típusgenerálás (SWAGGER) ... 16
12. Build és deployment ... 16
12.1. Fejlesztési parancsok ... 16
12.2. Docker Compose szolgáltatások ... 17
12.3. Turborepo konfiguráció ... 17
13. Tesztelés ... 17
14. Platformfüggő különbségek ... 18
15. Biztonsági megoldások ... 18

1. Projekt áttekintés

A Planova egy többplatformos naptáralkalmazás, amelynek frontend oldala két klienst tartalmaz:

- Web alkalmazás (Next.js + Tamagui)
- Mobil alkalmazás (Expo + React Native + Tamagui) *

Mindkét kliens ugyanazokat a megosztott csomagokat (packages) használja, ami biztosítja a kód újrafelhasználhatóságát és az egységes üzleti logikát. A projekt monorepo struktúrát alkalmaz pnpm workspace és Turborepo segítségével.

Az alkalmazás neve: Planova
Leírás: Felhasználó-központú naptáralkalmazás

2. Technológiai stack

2.1. Közös technológiák

- React 19 - UI könyvtár
- TypeScript - Típusbiztos JavaScript
- Tamagui - Univerzális UI keretrendszer (web + native)
- Zustand - Állapotkezelés (state management)
- TanStack React Query - Szerver-oldali állapotkezelés és cache
- Axios - HTTP kérések kezelése
- Zod - Séma validáció (űrlapok)
- React Hook Form - Űrlapkezelés
- SignalR (@microsoft/signalr) - Valós idejű értesítések (WebSocket)

2.2. Web alkalmazás (apps/web)

- Next.js (Turbopack) - React keretrendszer SSR/SSG támogatással
- @tamagui/next-plugin - Tamagui integráció Next.js-hez
- @tamagui/next-theme - Téma kezelés (sötét/világos mód)
- react-datepicker - Dátumválasztó komponens
- react-loading-indicators - Betöltési animációk
- Lucide Icons (@tamagui/lucide-icons) - Ikonkészlet

2.3. Mobil alkalmazás (apps/mobile) *

- Expo - React Native fejlesztői platform
- Expo Router - Fájl-alapú navigáció
- expo-secure-store - Biztonságos token tárolás
- React Navigation - Navigációs könyvtár (tab-ok, stack-ek)
- react-native-reanimated - Animációk
- react-native-gesture-handler - Gesztuskezelés
- react-native-screens - Natív képernyők optimalizálása

2.4. Fejlesztői eszközök

- pnpm - Csomagkezelő (v10.28.0)
- Turborepo - Monorepo build rendszer
- ESLint - Kódminőség-ellenőrzés
- openapi-typescript - API típusgenerálás Swagger-ből
- Docker / Docker Compose - Konténerizáció

3. Projekt struktúra

3.1. Monorepo felépítés

```
Vizsgaremek/
├── apps/
│   ├── web/ # Next.js webalkalmazás
│   ├── mobile/ # Expo mobil alkalmazás *
│   └── backend/ # .NET backend (C#)
├── packages/
│   ├── api/ # API hívások (axios)
│   ├── config/ # Tamagui konfiguráció és témák
│   ├── features/ # Megosztott feature komponensek
│   ├── hooks/ # Zustand store-ok és React Query hook-ok
│   ├── types/ # TypeScript típusdefiníciók + Swagger típusok
│   ├── ui/ # Újrafelhasználható UI komponensek
│   └── utils/ # Segédfüggvények és naptárlogika
├── scripts/ # Fejlesztői scriptek
├── docker-compose.yml # Docker Compose konfiguráció
├── pnpm-workspace.yaml # pnpm workspace konfiguráció
├── turbo.json # Turborepo konfiguráció
└── package.json # Gyökér csomag
```

3.2. Web alkalmazás struktúra (apps/web)

```
apps/web/
├── next.config.ts # Next.js + Tamagui konfiguráció
├── Dockerfile # Docker build a webalkalmazáshoz
├── package.json # Függőségek
├── public/
│   └── tamagui.css # Tamagui CSS kimenet
└── src/
    ├── app/
    │   ├── layout.tsx # Gyökér layout (HTML, Providers)
    │   ├── page.tsx # Főoldal (átirányít /sign-in-re)
    │   ├── globals.css # Globális CSS stílusok
    │   ├── sign-in/
    │   │   └── page.tsx # Bejelentkezési oldal
    │   ├── testpage/
    │   │   └── page.tsx # Tesztoldal
    │   └── (dashboard)/ # Védett route csoport
    │       ├── layout.tsx # Dashboard layout (SignalR + védelem)
    │       ├── calendar/
    │       │   └── page.tsx # Naptár főoldal
    │       └── profile/
    │           └── page.tsx # Profil oldal (fejlesztés alatt)
    └── components/
```

```
├── Providers.tsx # Alkalmazás provider-ek (téma, query, auth)
├── ProtectedRoute.tsx # Hozzáférés-védelmi wrapper
├── FullscreenView.tsx # Teljes képernyős elrendezés komponens
├── Navbar/
│   ├── Navbar.tsx # Navigációs sáv (publikus + privát)
│   └── ThemeToggle.tsx # Sötét/világos mód váltó
└── CalendarPage/
    ├── Sidebar.tsx # Oldalsáv (naptárkezelés)
    ├── CalendarHeader.tsx # Naptár fejléc (navigáció, nézetek)
    ├── CalendarEntryDetail.tsx # Bejegyzés részletei
    ├── CalendarEntryHourView.tsx # Óra nézet bejegyzésekhez
    ├── ColorSelect.tsx # Szín választó
    ├── ContextMenu.tsx # Jobb-klikk kontextus menü
    ├── CreateCalendarForm.tsx # Naptár létrehozó űrlap
    ├── CreateCalendarEntryForm.tsx # Bejegyzés létrehozó űrlap
    ├── UpdateCalendarForm.tsx # Naptár szerkesztő űrlap
    └── CustomDialog.tsx # Egyedi dialógus ablak
```

3.3. Mobil alkalmazás struktúra (apps/mobile) *

```
apps/mobile/
├── app.json # Expo konfiguráció
├── babel.config.js # Babel konfiguráció
├── metro.config.js # Metro bundler konfiguráció
├── package.json # Függőségek
├── app/
│   ├── _layout.tsx # Gyökér layout (auth, providerek)
│   ├── index.tsx # Belépési pont (auth-alapú átirányítás)
│   ├── (guest)/ # Vendég útvonalak
│   │   ├── _layout.tsx # Tab navigáció (Home, Sign-In)
│   │   ├── home.tsx # Kezdőlap
│   │   └── sign-in.tsx # Bejelentkezés
│   └── (protected)/ # Védett útvonalak
│       ├── _layout.tsx # Tab navigáció (Calendar)
│       └── calendar.tsx # Naptár oldal
├── android/ # Android natív projekt
└── ...
```

3.4. Megosztott csomagok (packages/)

Minden csomag workspace hivatkozással érhető el: @repo/<csomag_név>

packages/api/ → @repo/api
packages/config/ → @repo/config
packages/features/ → @repo/features

packages/hooks/ → @repo/hooks
packages/types/ → @repo/types
packages/ui/ → @repo/ui
packages/utils/ → @repo/utils

4. Megosztott csomagok részletes leírása

4.1. @repo/api – API kommunikációs réteg

Az API csomag felelős a backend-del való minden HTTP kommunikációért.

Fájlok:

- api.ts - Axios instance konfigurálása, interceptorok
- auth.ts - Autentikációs végpontok (signUp, signIn, signOut, verify, refresh)
- calendar.ts - Naptár CRUD műveletek (getCalendar, createCalendar, updateCalendar, deleteCalendar)
- calendarEntry.ts - Naptárbejegyzés CRUD (getCalendarEntry, createCalendarEntry, updateCalendarEntry, deleteCalendarEntry)
- profile.ts - Profil műveletek (getProfile, updateProfile)
- notification.ts - Barát kérés küldése (sendFriendRequest)
- nativeTokenStorage.ts - Token tárolás kezelése (natív vs. web)

Axios instance jellemzők:

- Alap URL: http://localhost:8080
- Web: cookie-alapú hitelesítés (withCredentials: true)
- Natív: Bearer token hitelesítés (expo-secure-store)
- Automatikus token frissítés interceptorral (401-es hiba esetén)
- Kérés sorba állítás token frissítés közben (failedQueue)

Token kezelés:

- Web: böngésző cookie-kat használ (a szerver állítja be)
- Natív: expo-secure-store-ban tárolt accessToken és refreshToken
- Az initializeNativeTokenStorage() függvénnyel injektálható a SecureStore

4.2. @repo/hooks – Állapotkezelés (Zustand store-ok és React Query hook-ok)

Zustand store-ok:

useAuthStore
Állapot: isAuthorized, isLoading
Műveletek: verifyAuth() – token érvényesítés és automatikus refresh
Folyamat: verify() → hiba esetén refresh() → hiba esetén isAuthorized=false

useCalendarStore
Állapot: currentDate, selectedDate, viewType, checkedCalendarIds, tabletView, mobileView, hideSidebar
Nézet típusok: "month" | "multiweek" | "week" | "day"
Navigáció: decMonth/incMonth, decWeek/incWeek, decDay/incDay, decYear/incYear, resetToToday
Reszponzivitás: initResizeListener() – ablakméret változás figyelése (768px=mobil, 1024px=tablet)

useProfileStore
Állapot: locale (Intl.Locale), weekStartsOn, hour12
Perzisztencia: localStorage-ba menti a beállításokat
Naptár kezdőnapja: "monday" | "sunday"
Óra formátum: 12 órás vagy 24 órás

useContextMenuStore
Jobb-klikk kontextus menü kezelése
Állapot: position, display, contextMenuOpen, fieldType, date
Típusok: "cell" (üres cella) | "schedule" (meglévő bejegyzés)

useDialogStore
Felugró dialógus ablakok kezelése
Állapot: isDialogOpen, title, description, content (React node)

useNavbarStore
Navigációs sáv állapota (beállítások panel nyitva/zárva)

useNotificationStore
SignalR kapcsolat kezelése (valós idejű értesítések)
Állapot: connection, notifications, isConnected
SignalR Hub: /notifications végpont
Automatikus újracsatlakozás (.withAutomaticReconnect())
Web: cookie-kat használ | Natív: access token-t küld

React Query hook-ok:

useCalendars
Lekéri az összes naptárat (GET api/Calendar)
Cache: 5 perc staleTime
Query kulcs: ["myCalendars"]

useCalendarEntries
Lekéri a naptárak bejegyzéseit (párhuzamos kérések minden naptárnak)
Feltétel: myCalendarsQuery.data elérhetősége
Cache: 5 perc staleTime
Query kulcs: ["calendarEntries", calendarIds]

useProfile
Profil adatok lekérése

4.3. @repo/features – Megosztott feature komponensek

Hitelesítés:

- AuthCard: Toggle-özhető bejelentkezés/regisztráció kártya
- SignInForm: Bejelentkezési űrlap (email + jelszó, Zod validáció)
- SignUpForm: Regisztrációs űrlap (email + jelszó erős validációval)

Jelszó követelmények (SignUpForm):
- Minimum 8, maximum 128 karakter
- Legalább 1 kisbetű
- Legalább 1 nagybetű
- Legalább 1 szám
- Legalább 1 speciális karakter

Naptár:

- Calendar: Fő naptár komponens (havi, többhetes, heti, napi nézet)
- CalendarCell: Egyedi naptár cella komponens
- CalendarEntry: Naptárbejegyzés megjelenítése

4.4. @repo/ui – Újrafelhasználható UI komponensek

- StyledButton - Egységes stílusú gomb
- StyledInput - Egységes stílusú szövegbeviteli mező
- StyledTextArea - Egységes stílusú szövegterület
- SelectElement - Egyedi legördülő menü
- ColorIcon - Szín ikon megjelenítés
- ToggleGroupItemText - Toggle csoport szöveges elem

A Tamagui keretrendszer biztosítja, hogy ezek a komponensek mindkét platformon (web és natív) azonos kinézettel és viselkedéssel működnek.

4.5. @repo/types – Típusdefiníciók

- calendar.ts - Naptár típusok: CalendarViewType, CalendarCellProps, WeekStartDay, MonthLabel, WeekDayLabel stb.
- swagger.ts - Automatikusan generált típusok a backend Swagger/OpenAPI specifikációjából (openapi-typescript)

Nézet típusok (CalendarViewType):
"month" – havi nézet
"multiweek" – többhetes nézet (4 hét)
"week" – heti nézet
"day" – napi nézet

Naptár cella (CalendarCellProps):

- date: Date – a cella dátuma
- inCurrentMonth: boolean – az adott hónapba tartozik-e

4.6. @repo/utils – Segédfüggvények

Naptár logika:

- calendarGridFactory.ts - Naptár rács generálás különböző nézetekhez
  generateMonthCells() – 35/42 cella a havi nézethez
  generateMultiweekCells() – 28 cella (4 hét) a többhetes nézethez
  generateWeekCells() – 7 cella a heti nézethez
  generateDayCell() – 1 cella a napi nézethez
  generateGrid() – fő hívó függvény, hét számok szerinti csoportosítás
- dateMethods.ts - Dátum segédfüggvények
- Month.ts - Hónap osztály
- Week.ts - Hét osztály
- calendarEntryUtils.ts - Bejegyzés pozicionálási logika

Szín segédlet:

- colors.ts - getContrastFromHSLA(): HSLA szín alapján visszaadja, hogy fekete vagy fehér szövegszín szükséges a megfelelő kontraszthoz (YIQ képlet)

Platform detektálás:

- isNative.ts - isNative(): meghatározza, hogy React Native környezetben fut-e az alkalmazás

4.7. @repo/config – Tamagui konfiguráció

- tamagui.config.ts - Tamagui teljes konfigurációja (a defaultConfig v4-re épül)
- fonts.ts - Betűtípus beállítások
- themes.ts - Téma definíciók:
  Alap paletta: 12 szín sötét és világos módhoz (HSLA formátum)
  Accent paletta: Kiemelő színek (kék árnyalatok)
  Gyerek témák: warning (sárga), error (piros), success (zöld), info (kék), calendarColors (12 naptár szín)
  Árnyékok: lightShadows, darkShadows (6 szint)

5. Alkalmazás működése

5.1. Indítás és hitelesítés

Betöltési folyamat:

1. Az alkalmazás elindul, a Providers komponens betöltődik
2. Az useAuthStore.verifyAuth() meghívódik
3. Először verify() API hívás történik (token érvényesítés)
4. Ha sikertelen, refresh() hívás (token frissítés)
5. Ha a frissítés is sikertelen: isAuthorized = false
6. A betöltési állapot alatt egy animált OrbitProgress jelenik meg

Web navigáció:

- / → átirányítás /sign-in-re
- /sign-in → bejelentkezés/regisztráció oldal (ha autorizált → /calendar)
- /calendar → naptár oldal (védett)
- /profile → profil oldal (fejlesztés alatt)

Mobil navigáció: *

- index.tsx → auth ellenőrzés → (guest)/sign-in VAGY (protected)/calendar
- (guest) csoport: Home tab + Sign-In tab
- (protected) csoport: Calendar tab

5.2. Hitelesítési folyamat

Bejelentkezés (SignInForm):

1. Felhasználó megadja az email címet és jelszót
2. Zod validáció ellenőrzi a mező kitöltöttségét
3. signIn() API hívás (POST api/auth/sign-in)
4. Webre: a szerver cookie-kat állít be

5. Nativra: accessToken + refreshToken tárolódik SecureStore-ban
6. Sikeres bejelentkezés után: átirányítás a /calendar oldalra

Regisztráció (SignUpForm):

1. Email + jelszó validáció (erős jelszó követelmény)
2. signUp() API hívás (POST api/auth/sign-up)
3. Automatikus bejelentkezés a signIn() hívással
4. Átirányítás a /calendar oldalra

Kijelentkezés:

1. Token-ek törlése (SecureStore vagy cookie)
2. signOut API hívás (GET api/auth/sign-out)
3. Átirányítás a /sign-in oldalra (window.location.href)

5.3. Naptár nézetek

A naptár négy különböző nézetet támogat:

Havi nézet ("month"):
- 5 vagy 6 hetes rács (35 vagy 42 cella)
- Hét számok az oldalsávban
- Előző/következő havi napok halványítva
- Görgős navigáció (mousewheel)

Többhetes nézet ("multiweek"):
- 4 hetes rács (28 cella)
- Az aktuális hét a kiválasztott dátum alapján

Heti nézet ("week"):
- 7 nap, óra-alapú rács (0-23 óra)
- Oldalra görgethető tartalom

Napi nézet ("day"):
- 1 nap, 24 órás rács
- Részletes időbeosztás megjelenítés

Navigáció:

- Előző/következő gombokkal (nyílak)
- Egérgörgővel (havi nézetben)
- "Ma" gombbal (resetToToday)
- Nézet típus váltó (legördülő menü)

5.4. Oldalsáv (Sidebar)

Az oldalsáv tartalmazza:

- DatePicker: mini naptár a dátumválasztáshoz
- Naptárlista: a felhasználó naptárai checkbox-okkal
  - Naptár szín jelzése
  - Naptár szerkesztése (név, szín módosítás)
  - Naptár törlése
- Új naptár létrehozása (CalendarPlus ikon + CreateCalendarForm)
- Reszponzív: tablet nézetben elrejthető (hamburger menü)

5.5. Naptárbejegyzések

Bejegyzés létrehozása:

1. Jobb-klikk egy cellára → kontextus menü → "New Event"
2. Vagy a napi/heti nézetben "+" gomb
3. CreateCalendarEntryForm megnyílik dialógusban
4. Mezők: naptár kiválasztás, kategória, név, leírás, egész napos, kezdő dátum, befejező dátum, szín
5. Zod validáció után API hívás

Bejegyzés kategóriák (EntryCategory):

- None (Nincs)
- Task (Feladat)
- Event (Esemény)
- BirthDay (Születésnap)
- Anniversary (Évforduló)

Naptárbejegyzés színek:
12 előre definiált szín áll rendelkezésre (color1-color12), amelyeket a calendarColors téma definiál.

5.6. Kontextus menü

A jobb-klikk kontextus menü a ContextMenuStore-on keresztül működik:

- Cella típus ("cell"): Új bejegyzés létrehozása
- Bejegyzés típus ("schedule"): Bejegyzés szerkesztése/törlése
- Pozíció automatikus igazítása az ablak széleihez
- Bal klikk a menün kívülre bezárja

5.7. Dialógus ablakok

A CustomDialog komponens:

- Modal megjelenítés (háttér elsötétítés)
- Testreszabható cím, leírás és tartalom
- Animált be/kimenet
- Maximális magasság korlátozás (90vh)
- Globálisan is használható a useDialogStore-on keresztül

5.8. Valós idejű értesítések (SignalR)

- A dashboard layout indítja a SignalR kapcsolatot
- Hub végpont: /notifications
- Automatikus újracsatlakozás
- Web: cookie-alapú hitelesítés | Natív: Bearer token
- ReceiveNotification esemény fogadása
- Értesítések listában tárolódnak (notifications tömb)

6. Navigációs sáv (Navbar)

Két változat létezik az autorizáció állapotától függően:

Publikus Navbar (nem bejelentkezett felhasználó):

- Planova logó + alkalmazás neve
- "Sign In / Sign Up" gomb
- Téma váltó gomb (kör alakú)

Privát Navbar (bejelentkezett felhasználó):

- Hamburger menü (oldalsáv nyitás/zárás)
- Planova logó + alkalmazás neve
- Avatar (kattintásra beállítások panel)

Beállítások panel (Popover):

- Téma váltó (sötét/világos mód switch)
- Hét kezdőnapja (hétfő/vasárnap toggle)
- Óra formátum (12 órás/24 órás toggle)
- Kijelentkezés gomb

7. Reszponzív működés

A webalkalmazás reszponzív tervezésű:

- Desktop (> 1024px): Teljes nézet, oldalsáv látható
- Tablet (768px – 1024px): Oldalsáv alapértelmezetten elrejtve, hamburger menüvel megnyitható
- Mobil (< 768px): Mobil nézet, egyszerűsített elrendezés

Az ablakméret figyelése az useCalendarStore.initResizeListener() függvényen keresztül történik, amely resize event listener-t regisztrál.

8. Téma rendszer

Az alkalmazás sötét és világos módot támogat.

Szín paletta felépítés (12 szint):

1. Background (háttér)
2. Subtle background (enyhe háttér)
3. Inactive surface (inaktív felület)
4. Default surface (alapértelmezett felület)
5. Hover surface (hover állapot)
6. Active surface (aktív felület)
7. Default border (alapértelmezett szegély)
8. Hover border (hover szegély)
9. Muted fill (tompított kitöltés)
10. Strong fill (erős kitöltés)
11. Secondary text (másodlagos szöveg)
12. Primary text (elsődleges szöveg)

Speciális témák:

- accent: Kiemelő szín (Google Blue: #1A73E8)
- warning: Figyelmeztetés (sárga - amber)
- error: Hiba (piros - red)
- success: Siker (zöld - green)
- info: Információ (kék - blue)
- calendarColors: 12 naptár szín

Téma tárolás:

- Web: NextThemeProvider kezeli (automatikus rendszerpreferencia detektálás)
- Natív: useColorScheme() alapján (rendszer téma)

Globális CSS változók:
--navbar-height: 60px (navigációs sáv magassága)

--scrollbar-width: 15px (görgetősáv szélessége)
Betűtípus: Roboto (Google Fonts)

9. Provider struktúra

9.1. Web (Providers.tsx)

```
Providers (legkülső)
└── NextThemeProvider (téma perzisztencia)
└── InnerProviders
├── auth verify (hitelesítés ellenőrzés indításkor)
├── TamaguiProvider (Tamagui UI keretrendszer)
│   └── QueryClientProvider (React Query cache)
│   └── MainView (kontextus menü + dialógus kezelés)
│   ├── ContextMenu (globális jobb-klikk menü)
│   ├── Navbar (navigációs sáv)
│   └── {children} (oldal tartalma)
```

9.2. Mobil (_layout.tsx) *

```
QueryClientProvider (React Query cache)
└── TamaguiProvider (Tamagui UI)
└── ThemeProvider (React Navigation téma)
└── Stack (Expo Router navigáció)
├── (guest) (vendég útvonalak)
└── (protected) (védett útvonalak)
```

10. API végpontok (frontend oldal)

A frontend az alábbi backend API végpontokat használja:

Autentikáció:
POST api/auth/sign-up - Regisztráció
POST api/auth/sign-in - Bejelentkezés
GET api/auth/sign-out - Kijelentkezés
GET api/auth/verify - Token érvényesítés
GET api/auth/refresh - Token frissítés

Naptár:
GET api/Calendar - Naptárak lekérdezése
POST api/Calendar - Naptár létrehozása
PUT api/Calendar/{id} - Naptár módosítása
DELETE api/Calendar/{id} - Naptár törlése

Naptárbejegyzés:
GET api/CalendarEntry/{id} - Bejegyzések lekérdezése (szűrhető dátummal)
POST api/CalendarEntry - Bejegyzés létrehozása
PUT api/CalendarEntry/{id} - Bejegyzés módosítása
DELETE api/CalendarEntry/{id} - Bejegyzés törlése

Profil:
GET api/Profile - Profil lekérdezése
PUT api/Profile/{id} - Profil módosítása

Barát kérés:
POST api/friend-request - Barát kérés küldése

SignalR Hub:
/notifications - Valós idejű értesítések (WebSocket)

11. Típusgenerálás (SWAGGER)

A projekt automatikusan generálja a TypeScript típusokat a backend Swagger specifikációjából:

Parancs: pnpm generate-types
Forrás: http://localhost:5273/swagger/v1/swagger.json
Kimenet: packages/types/src/swagger.ts

Ez biztosítja, hogy a frontend típusai mindig szinkronban legyenek a backend API-val. A generált típusok a components["schemas"] névtéren keresztül érhetők el, például:

- components["schemas"]["SignInRequestDTO"]
- components["schemas"]["GetCalendarDTO"]
- components["schemas"]["CreateCalendarEntryDTO"]
- components["schemas"]["GetProfileDTO"]
- components["schemas"]["EntryCategory"]
- components["schemas"]["TokenResponseDTO"]

12. Build és deployment

12.1. Fejlesztési parancsok

pnpm dev - Interaktív dev script (backend + frontend)
pnpm dev:all - Összes service indítása párhuzamosan
pnpm web:dev - Csak a webalkalmazás indítása
pnpm mobile:dev - Csak a mobil alkalmazás indítása

pnpm backend:dev - Csak a backend indítása
pnpm generate-types - Swagger típusgenerálás
pnpm build - Teljes build (Turborepo)
pnpm test - Tesztek futtatása
pnpm reinstall - Csomagok újratelepítése
pnpm clean:packages - Csomagok törlése
pnpm clean:android - Android build törlése

12.2. Docker Compose szolgáltatások

db - PostgreSQL 17.6 adatbázis (5432-es port)
backend - .NET backend API (8080-as port)
typegen - Swagger típusgenerátor (egyszeri futás)
web - Next.js webalkalmazás (3000-es port)

Indítás:

./script/docker-start.ps1 → script futtatása a **projekt mappából powershellben**

Build folyamat (web Dockerfile):

1. deps szakasz: pnpm install --shamefully-hoist
2. build szakasz: Next.js build Turbopack-kel
3. runner szakasz: Minimális production image

12.3. Turborepo konfiguráció

dev: cache kikapcsolva, persistent
build: dependsOn ["^build"], kimenet: .next/**, dist/**, build/**
start: dependsOn ["build"], persistent
test: cache kikapcsolva

13. Tesztelés

A frontend tesztek a következő fájlokban találhatók:

Zustand store tesztek:

- packages/hooks/src/useAuthStore.test.ts
- packages/hooks/src/useCalendarStore.test.ts
- packages/hooks/src/useContextMenuStore.test.ts
- packages/hooks/src/useProfileStore.test.ts

Utility tesztek:

- packages/utils/src/colors.test.ts
- packages/utils/src/calendar/calendarGridFactory.test.ts
- packages/utils/src/calendar/dateMethods.test.ts
- packages/utils/src/calendar/Month.test.ts
- packages/utils/src/calendar/Week.test.ts

Teszt futtatás: pnpm test

14. Platformfüggő különbségek

Jellemző	Web	Mobil *
Keretrendszer	Next.js	Expo (React Native)
Navigáció	Next.js App Router	Expo Router (Tabs)
Téma kezelés	NextThemeProvider	React Navigation Theme
Token tárolás	HTTP cookie-k	expo-secure-store
Auth header	Cookie (automatikus)	Bearer token (header)
Build rendszer	Turbopack	Metro bundler
Platform detektálás	isNative() = false	isNative() = true
UI framework	Tamagui (web)	Tamagui (native)
Deployment	Docker + Next.js	Expo build
Scrollbar	Egyedi CSS scrollbar	Natív scrollbar

15. Biztonsági megoldások

- JWT alapú hitelesítés (access + refresh token)
- Automatikus token frissítés 401-es válasz esetén
- Védett útvonalak (ProtectedRoute komponens)
- Kérések sorba állítása token frissítés közben
- Auth végpontok kizárása a refresh logikából
- Natív: SecureStore (titkosított tárolás)
- Web: HttpOnly cookie-k (XSS védelem)
- Zod séma validáció minden űrlapon (kliens-oldali)
- Erős jelszó követelmények regisztrációnál